

SB STOCKS

Full Stack Development with MERN
Project Technical & System Documentation

Project Title:	SB Stocks — Virtual Stock Trading & Portfolio Simulator
Team Member:	Polisetty Eekshitha (Full Stack Developer)
Academic Track:	B.Tech Computer Science (3rd Year Undergraduate)
Documentation:	System Architecture, Setup Guide, API Specs, and Codebase Directory

1. Introduction

SB Stocks is a professional-grade web application designed for stock market enthusiasts, students, and active traders who want to practice and improve their trading skills without any financial risk. Leveraging the MERN (MongoDB, Express, React, Node.js) technology stack, this platform provides interactive, virtual mock-trading features powered by real-time data, historical asset trends, and an intuitive portfolio management engine.

This document serves as the comprehensive technical guide, architecture layout, setup documentation, and API reference manual for the project.

Skills & Technologies Applied

HyperText Markup Language (HTML)

Cascading Style Sheets (CSS)

JavaScript (ES6+)

React.js (Frontend)

Node.js (Runtime)

Express.js (API Layer)

MongoDB (Database)

CSS Frameworks & Modules

2. Project Overview

Core Concept:

SB Stocks is a user-friendly web app built for stock market enthusiasts who want to practice and improve their trading skills without any financial risk. The platform provides paper trading features powered by real-time US stock market data, historical trends, and a wide range of stock listings, making it easier for users to make informed decisions.

With a simple registration and secure login process, users can access their personalized dashboard to simulate buying and selling stocks using virtual funds, track their portfolio's performance, and analyze strategies in a realistic environment. Whether you are a beginner exploring the basics of trading or an experienced trader looking to test new approaches, SB Stocks offers a seamless and educational experience. By combining a clean interface with comprehensive stock data and robust portfolio management tools, it ensures that every trading journey is both insightful and engaging, helping users refine strategies and build confidence in the world of stock trading.

Key Modules & System Features

- **Real-time Mock Trading:** Simulated order execution (buy/sell limit or market-style orders) processed instantaneously using an isolated portfolio ledger.
- **Unified Analytics & Dashboard:** User-specific analytical dashboards illustrating current holdings, active capital distribution, available virtual funds, and real-time performance tracking.
- **Dynamic Security Layering:** Secured session contexts backed by JWT authentication payloads and Bcrypt cryptographic password protections.

3. System Architecture & Database Design

The platform utilizes a modern distributed client-server architecture to optimize load times and separate structural presentation layouts from data execution models.

| Mongoose Schema Layouts

- **User (`User.js`)**: Handles account credentials, password hashing salts, default virtual cash allocations (e.g., \$100,000 baseline funds), and metadata arrays.
- **Stock (`Stock.js`)**: Configures asset collections, global tickers, company information, current market prices, and historical records.
- **Portfolio (`Portfolio.js`)**: Maps users to individual asset holdings, storing averages, aggregate holding quantities, and cumulative transaction performance.

4. Setup Instructions

| System Prerequisites

- Node.js (v18.x or higher recommended)
- MongoDB Atlas Account (fully-provisioned cloud tier)
- Package Manager (NPM or Yarn)

| Configuration Files

Create a `.env` file directly inside the root of your backend directory containing the following verified connection configuration:

```
PORT=5000
MONGO_URI=mongodb+srv://sbstocks:sbstocks12345@sbstocks.18i66xr.mongodb.net/sbstocks?
retryWrites=true&appName=sbstocks
JWT_SECRET=sbstockssecretkey
```

| Installation and Local Startup

1. Server (Backend) Deployment:

```
cd backend
npm install
node server.js
```

2. Client (Frontend) Startup:

```
cd frontend
npm install
npm run start (or npm run dev depending on custom scripts)
```

3. Database Seeding:

```
cd backend
node seed/stockSeeder.js
```

5. Detailed Project Directory Structure

Below is the precise file mapping of the workspace as implemented in the project repository:

A. Backend Structure

```
backend/
├─ config/
│   └─ db.js # Mongo DB database connection wrapper
├─ controllers/
│   ├── authController.js # Session handshakes & tokens
│   ├── dashboardController.js # Financial statistics aggregation
│   ├── portfolioController.js # Transaction ledger updates
│   ├── stockController.js # Live price lists and updates
│   └─ userController.js # Profile details and metadata
├─ middleware/
│   └─ authMiddleware.js # Token parsing & access rules
├─ models/
│   ├── Portfolio.js # Trade statistics schema
│   ├── Stock.js # Global listings model
│   └─ User.js # Profile credentials schema
├─ routes/
│   ├── authRoutes.js
│   ├── dashboardRoutes.js
│   ├── portfolioRoutes.js
│   ├── stockRoutes.js
│   └─ userRoutes.js
├─ seed/
│   └─ stockSeeder.js # Populates original market lists
├─ utils/ # Custom helper modules
├─ .env # Backend settings configuration
├─ package.json # Backend dependency package ledger
└─ server.js # Entry script & server init engine
```

B. Frontend Structure

```
frontend/
├─ public/
└─ src/
    ├── assets/ # Static images & platform brand icons
    ├── components/
    │   ├── AuthLayout.css # Design styles for authentication layouts
    │   ├── AuthLayout.jsx # Shared form layout wrap
    │   ├── DashboardNavbar.css
    │   ├── DashboardNavbar.jsx # Navigation headers for dashboard context
    │   ├── Navbar.css
    │   ├── Navbar.jsx # General application Navbar layout
    │   ├── Sidebar.css
    │   ├── Sidebar.jsx # Sidebar dashboard layout navigation
    │   └─ StatCard.jsx # Component displays for user holdings stats
    ├── context/ # Global state providers
    └─ hooks/ # Reusable react hook implementations
```

```
├─ pages/
|   ├─ Dashboard.css / .jsx # Performance summaries
|   ├─ Home.css / .jsx      # Main landing layout
|   ├─ Login.css / .jsx     # Secure entrance forms
|   ├─ Portfolio.css / .jsx # User transaction positions
|   ├─ Profile.css / .jsx   # Individual settings views
|   ├─ Register.css / .jsx  # Account creation interfaces
|   └─ Stocks.css / .jsx    # Asset market panels
├─ services/                # API Client calls
|   ├─ authService.js
|   ├─ dashboardService.js
|   ├─ portfolioService.js
|   ├─ stockService.js
|   └─ userService.js
├─ App.js                  # Top React container with navigation
├─ index.js                # Target entry render point
└─ package.json            # Client packages and libraries
```

6. API Specifications

All endpoints process requests relative to the global gateway base URL: `http://localhost:5000/api`.

Route Path	Method	Access Level	Functional Description
<code>/auth/register</code>	POST	Public	Creates unique user credentials. Initializes starting cash ledger.
<code>/auth/login</code>	POST	Public	Verifies passwords against hashed profiles and delivers JWT.
<code>/stocks</code>	GET	Protected	Returns active inventory/market stock lists and historic patterns.
<code>/portfolio</code>	GET	Protected	Fetches current individual holdings, buy-costs, and metrics.
<code>/portfolio/trade</code>	POST	Protected	Simulates buying and selling properties, altering cash balances.

7. System Validation & QA Strategy

- **Unit Tests:** Individual testing targeting transaction calculators inside controllers to prevent negative cash ledger limits.
- **Integration Tests:** Verifying API authorization header filters correctly reject requests missing authentic bearer JWT markers.
- **End-to-End Evaluation:** Complete simulation flows from user onboarding, executing multiple limit trades, observing immediate updates in dashboard state, and evaluating performance trends.